SOLUTION DESIGN AND IMPLEMENTATION DOCUMENT

Intelligent Account Servicing Workflow (IASW)

1. EXECUTIVE SUMMARY

The Intelligent Account Servicing Workflow (IASW) is an AI-powered system designed to automate the verification and processing of Aadhaar-based name change requests in banking operations. The system employs a multi-agent architecture powered by Large Language Models (LLMs) to analyse identity document, calculate confidence scores, and generate compliance summaries.

KEY DELIVERABLES:
- Automated document verification using OCR and Vision AI
- Multi-dimensional confidence scoring for name change requests
- Human-in-the-Loop (HITL) approval workflow
- Audit trail and compliance documentation
- RPS (Real-time Platform System) integration with authorization controls

BUSINESS VALUE:
- Reduces manual review time by 60-70%
- Standardizes verification criteria across all requests
- Maintains regulatory compliance through documented audit trails
- Provides transparent decision-making with AI-generated justifications
- Ensures human oversight before critical system updates

TECHNOLOGY HIGHLIGHTS:
- 4 specialized AI agents orchestrated via LangChain
- Vision LLM for direct document image analysis
- Groq API for fast, cost-effective LLM inference
- Fallback mechanisms for offline/alternative processing
- HitL enforcement preventing autonomous AI decisions on banking updates

2. PROBLEM UNDERSTANDING & SCOPE

2.1 PROBLEM STATEMENT

Banking institutions process thousands of name change requests annually, each requiring manual verification of identity documents. This manual process suffers from:

1. INCONSISTENCY: Different reviewers may interpret the same document differently
2. SCALABILITY: Manual review cannot scale with request volume
3. ERROR RATE: Human fatigue leads to missed details
4. COMPLIANCE: Ensuring consistent application of verification rules
5. AUDIT: Maintaining comprehensive audit trails for regulatory compliance

2.2 SCOPE OF IMPLEMENTATION

INCLUDED:
- Aadhaar card-based name change verification
- OCR and Vision AI document extraction
- Multi-factor confidence scoring
- AI-generated compliance summaries
- Human checker review interface

- HITL approval workflow for banking system updates
- Complete audit logging

EXCLUDED:
- Other identity document types (PASSPORT, PAN, VOTER ID)
- Name change due to marriage/divorce legal proceedings
- Bulk/batch request processing
- Multi-document cross-referencing
- Real-time banking system integration (mock only for prototype)

2.3 USE CASE SPECIFICATIONS

PRIMARY USE CASE:
Customer submits a name change request with their Aadhaar card. The system automatically extracts document data, calculates confidence scores, and generates a recommendation. A human checker reviews the AI findings and makes the final approval/rejection decision.

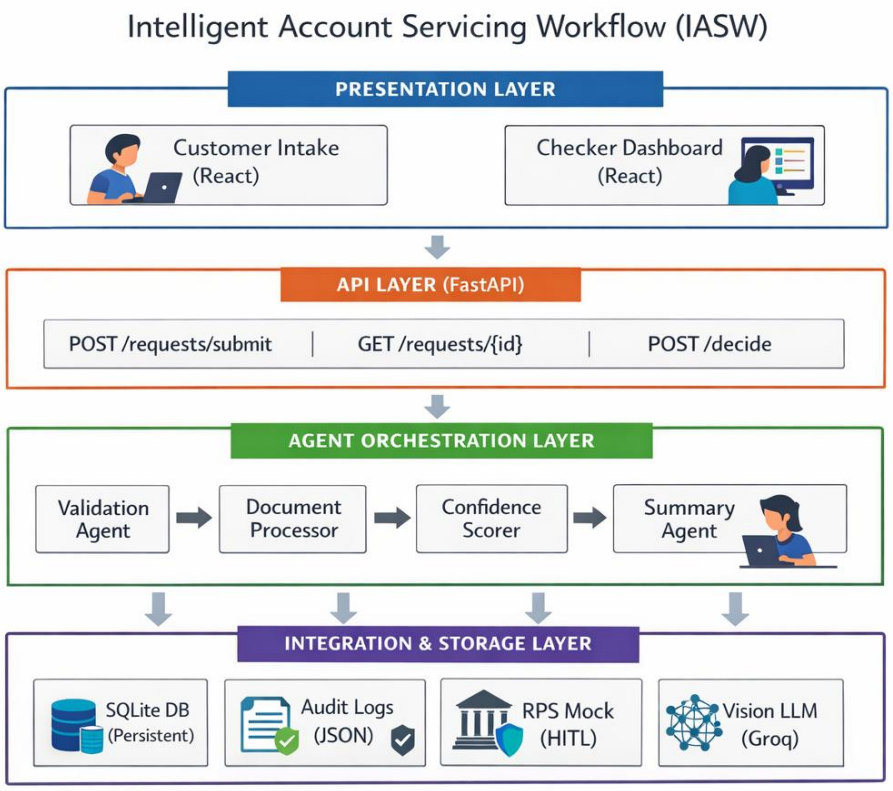
SECONDARY USE CASES:
- Request without document submission (lower confidence)
- Document with partial data extraction
- Multiple requests from same customer
- Request status tracking and history

2.4 SUCCESS CRITERIA

- AI recommendation accuracy: Target 90%+ alignment with human checker decisions
- Processing time: < 30 seconds per request (excluding human review)
- Fallback success: 100% of requests processed even without LLM availability
- Audit completeness: Every state transition logged with timestamps
- HITL compliance: Zero unauthorized RPS calls

3. SOLUTION ARCHITECTURE
3.1 HIGH-LEVEL ARCHITECTURE

The solution follows a modular, layered architecture:



3.2 DATA FLOW ARCHITECTURE

REQUEST FLOW:
1. Customer submits via frontend → API receives Form data + File upload
2. API passes to AIPipeline orchestrator
3. Pipeline calls agents sequentially with data transformations
4. Each agent logs activity to audit service
5. Final state persisted to SQLite database
6. Human checker retrieves via GET endpoint
7. Checker decision triggers RPS update (if approved)

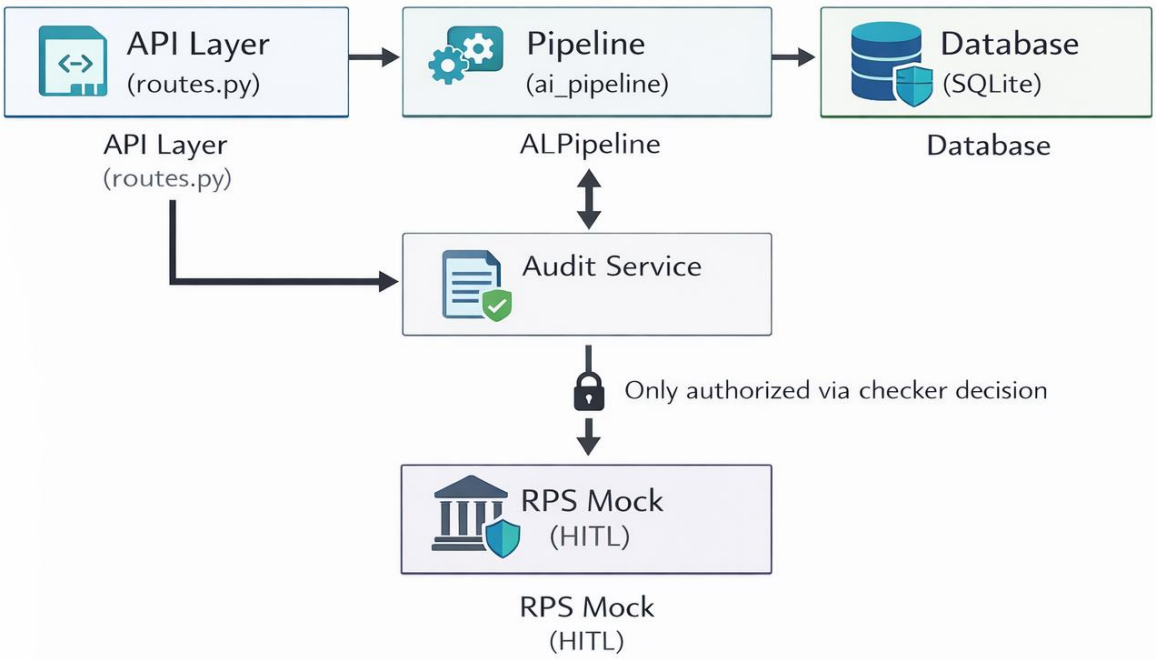
DATA TRANSFORMATIONS:

- Input: Form fields (old_name, new_name, DOB, Aadhaar) + Document (PDF/Image)

- Processing: Base64 encoding → OCR → Structured JSON → Scores → Summary

- Output: Request record with extracted_data, confidence_score, ai_summary

3.3 COMPONENT INTERACTIONS



3.4 DATABASE SCHEMA

REQUESTS TABLE:

- request_id (UUID, PRIMARY KEY)
- customer_id (VARCHAR)
- old_name, new_name (VARCHAR)
- date_of_birth, aadhar_number (VARCHAR, NULLABLE)
- status (ENUM: DRAFT, AI_PROCESSING, FAILED, AI_VERIFIED_PENDING_HUMAN, APPROVED, REJECTED, RPS_UPDATED)
- extracted_data (JSON)
- confidence_score (JSON)
- ai_summary (TEXT)
- ai_recommendation (VARCHAR)
- checker_decision, checker_id (VARCHAR, NULLABLE)
- rejection_reason (TEXT, NULLABLE)
- rps_reference (VARCHAR, NULLABLE)
- created_at, updated_at (TIMESTAMP)

AUDIT_LOGS TABLE:

- log_id (INTEGER, PRIMARY KEY)
- request_id (VARCHAR, FOREIGN KEY)
- event_type (VARCHAR)
- event_data (JSON)
- timestamp (TIMESTAMP)

3.5 SECURITY ARCHITECTURE

HITL ENFORCEMENT:

- RPSMockService._enforce_authorization(checker_id)
- Raises PermissionError if checker_id is None
- Only /checker/decide endpoint can provide valid checker_id
- AI agents cannot bypass this boundary

CORS CONFIGURATION:

- Allow all origins for prototype
- Allow credentials and all methods/headers
- Production should restrict to known frontend domains

3.6 DEPLOYMENT ARCHITECTURE

For Prototype:

- Backend: Local Python process (uvicorn)
- Frontend: Local Node process (Vite dev server)
- Database: SQLite file in data/ directory
- Logs: JSON files in data/logs/ directory

For Production (Recommended):

- Backend: Containerized (Docker) with Gunicorn/Uvicorn workers
- Frontend: Static hosting (Vercel/Netlify/CDN)
- Database: Managed SQLite or PostgreSQL
- Monitoring: Prometheus + Grafana for metrics

4. AGENT DESIGN & PROMPT ENGINEERING

4.1 AGENT OVERVIEW

Four specialized agents work in sequence, each designed with:

- LLM-powered processing (primary)
- Rule-based fallback (secondary)
- Structured output via Pydantic schemas
- Comprehensive logging

4.2 VALIDATION AGENT

PURPOSE: First gatekeeper ensuring request data integrity

DESIGN:

```
class ValidationAgent:
    - Uses ChatGroq with llama-3.3-70b-versatile
    - LLM chain: VALIDATION_PROMPT → LLM → JsonOutputParser
    - Rule-based fallback for offline scenarios
```

SYSTEM PROMPT:

```
"""
Validate identity verification request:
1. Both old_name and new_name are present and valid (letters, spaces only)
2. Names are reasonable length (2-100 chars)
3. New name is different from old name
4. DOB format is reasonable if provided
5. Aadhaar format is valid 12 digits if provided

Return JSON: is_valid, errors[], warnings[], explanation
"""
```

RULE-BASED LOGIC:

```
- NAME_PATTERN = r'^[a-zA-Z\s\-\.]+$'
- MIN_LEN, MAX_LEN = 2, 100
- ADHAR_PATTERN = r'^\d{12}$|^\d{4}\s?\d{4}\s?\d{4}$'
```

OUTPUT SCHEMA (Pydantic):

```
class ValidationResult(BaseModel):
```

is_valid: bool

errors: list

warnings: list

explanation: str

4.3 DOCUMENT PROCESSOR AGENT

PURPOSE: Extract structured data from identity documents

DESIGN:

class DocumentProcessor:

- Primary: Vision LLM (meta-llama/llama-4-scout-17b-16e-instruct)
- Secondary: LLM + OCR text extraction
- Tertiary: Regex-based extraction

VISION PROMPT:

"""

You are an expert document analyst specializing in Indian identity documents.

Examine the uploaded image and extract structured information.

Return JSON with ONLY these fields:

- name: Full legal name as it appears on the document
- date_of_birth: Date of birth (try to normalize to DD/MM/YYYY format)
- aadhar_number: Aadhaar card number (12 digits)
- raw_text: The complete text visible in the document image
- document_authentic: true if document appears genuine, false if suspicious/forged

Look carefully for signs of tampering, poor image quality, or inconsistency.

Return null for fields you cannot confidently determine.

"""

OCR EXTRACTION PROMPT:

"""

You are an expert document analyst specializing in Indian identity documents.

Extract structured information from the provided document text.

Return JSON with ONLY these fields:

- name: Full legal name as it appears on the document
- date_of_birth: Date of birth (try to normalize to DD/MM/YYYY format)
- aadhar_number: Aadhaar card number (12 digits, may have spaces)
- raw_text: The complete text extracted from the document
- document_authentic: true if document appears genuine, false if suspicious/forged

"""

FALLBACK REGEX PATTERNS:

- Name: r"(?:Name|Beneficiary|Holder)[:\s]+([A-Z][a-z]+(?:\s+[A-Z][a-z]+)+)"
- DOB: r"(?:DOB|Date of Birth|D\.O\.B)[:\s]*(\d{1,2}[-/]\d{1,2}[-/]\d{2,4})"
- Aadhaar: r"(?:Aadhaar|Aadhar|Adhar)[:\s]*(?:No\.?)?[:\s]*(\d{4}\s?\d{4}\s?\d{4})"

OUTPUT SCHEMA (Pydantic):

```
class DocumentData(BaseModel):
    name: Optional[str]
    date_of_birth: Optional[str]
    aadhar_number: Optional[str]
    raw_text: Optional[str]
    document_authentic: bool = True

@dataclass
class ExtractionResult:
    name: Optional[str]
    date_of_birth: Optional[str]
    aadhar_number: Optional[str]
    raw_text: str
    forgery_flag: bool
    document_authentic: bool = True
```

4.4 CONFIDENCE SCORER AGENT

PURPOSE: Calculate verification confidence and generate recommendation

DESIGN:

class ConfidenceScorer:

- Uses ChatGroq with llama-3.3-70b-versatile
- LLM chain for intelligent scoring
- Rule-based chain for offline fallback
- difflib.SequenceMatcher for fuzzy matching

SYSTEM PROMPT:

"""

You are an expert at verifying Aadhaar card NAME CHANGE requests.

CRITICAL: Determine if this is a VALID name change request.

NAME CHANGE SCORE (0-100):

- HIGH (85-100): Extracted name matches OLD name but NOT new name (valid name change) OR extracted matches both (spelling correction)
- MEDIUM (50-84): Partial match, needs review
- LOW (0-49): Extracted name matches NEW name only (invalid - document already updated)

DOCUMENT_TO_OLD_MATCH (0-100): How well extracted name matches OLD name. Higher = better identity proof.

DOCUMENT_TO_NEW_MATCH (0-100): How well extracted name matches NEW name. For valid name change: this should be LOW.

Return JSON with all scores and clear reasoning.

"""

SCORING DIMENSIONS:

1. name_change_request_score (0-100): Is this a valid name change request?
2. document_to_old_match (0-100): Document matches OLD name

3. document_to_new_match (0-100): Document matches NEW name

4. dob_match (0-100): DOB verification

5. adhar_match (0-100): Aadhaar number validation

6. doc_auth (0-100): Document authenticity

7. overall (0-100): Weighted composite score

OVERALL CALCULATION (Rule-based fallback):

- name_change_request_score: 30%
- document_to_old_match: 25%
- dob_match: 15%
- adhar_match: 15%
- doc_auth: 15%

RECOMMENDATION LOGIC:

- >= 85: "APPROVE"
- >= 70: "APPROVE_WITH_CAUTION"
- >= 50: "MANUAL_REVIEW"
- < 50: "REJECT"

OUTPUT SCHEMA (Pydantic):

```
class ConfidenceScores(BaseModel):
    name_change_request_score: int
    document_to_old_match: int
    document_to_new_match: int
    dob_match: int
    adhar_match: int
    doc_auth: int
    overall: int
    reasoning: str
```

4.5 SUMMARY AGENT

PURPOSE: Generate human-readable compliance summary

DESIGN:

class SummaryAgent:

- Uses ChatGroq with llama-3.3-70b-versatile
- StrOutputParser for natural language output
- Rule-based summary fallback

SYSTEM PROMPT:

"""

You are a compliance officer reviewing NAME CHANGE REQUESTS for Aadhaar cards.

IMPORTANT CONTEXT:

- Aadhaar document reflects the MOST RECENT (UPDATED) name of the individual.
- Therefore, the extracted name from the document is expected to match the NEW NAME (not the old name).
- A low match between document and OLD NAME is NOT an error if the document strongly matches the NEW NAME.

DECISION LOGIC (STRICTLY FOLLOW):

1. VALID NAME CHANGE PATTERN (APPROVE):
   - Document name matches NEW NAME strongly (high document_to_new_match)
   - Document name has partial or low similarity with OLD NAME
   - This indicates the person has already updated their name → VALID CASE

2. REJECTION CONDITIONS:
   - Document does NOT match NEW NAME
   - OR document matches neither OLD nor NEW name
   - OR authenticity / Aadhaar / DOB scores are low

3. PRIORITY RULE:
   - document_to_new_match is MORE IMPORTANT than document_to_old_match
   - High new match + low old match = EXPECTED and VALID

Generate a structured summary with these sections:

```
1. REQUEST DETAILS:
   - Type: Name Change
   - Old Name
   - New Name
   - DOB

2. DOCUMENT VERIFICATION:
   - Extracted name and fields from Aadhaar

3. NAME CHANGE ANALYSIS:
   - Compare OLD vs EXTRACTED
   - Compare NEW vs EXTRACTED
   - Explicitly explain why this matches a valid name change pattern or not

4. FLAGS/CONCERNS:
   - Mention only real risks (fraud, mismatch, low confidence)

5. RECOMMENDATION:
   - Must be either APPROVE or REJECT
   - Justify using the decision logic above
   - Explicitly reference: document_to_new_match, document_to_old_match,
     overall confidence
"""
```

4.6 AGENT ORCHESTRATION

Pipeline Execution (AIPipeline.process_request):

```
1. VALIDATION_AGENT.validate()
   └→ Returns (status, message)
      - If INVALID: Return FAILED status immediately

2. DOCUMENT_PROCESSOR.process_document()
```

└→ Returns (success, ExtractionResult)

- Extracts: name, DOB, Aadhaar, forgery_flag

3. CONFIDENCE_SCORER.score()

└→ Returns confidence_scores dict

- Calculates all dimensions + overall
- Generates recommendation

4. SUMMARY_AGENT.generate_summary()

Returns ai_summary string

- Human-readable compliance report

5. Persist to database with status = AI_VERIFIED_PENDING_HUMAN

5. ASSUMPTIONS, CONSTRAINTS & LIMITATIONS

5.1 ASSUMPTIONS

DOCUMENT ASSUMPTIONS:

- Aadhaar cards are the primary identity document
- Document images are clear and readable (minimum 300 DPI recommended)
- Documents are in standard Aadhaar format (UIDAI standard)
- No multi-page or multi-document submissions

PROCESSING ASSUMPTIONS:

- Groq API is available and responsive (< 10s latency)
- SQLite database can handle expected transaction volume
- Audit log rotation not required for prototype
- Single-instance deployment (no horizontal scaling)

USER ASSUMPTIONS:

- Checkers are authenticated and authorized
- Checkers understand the decision criteria
- Customer submissions are made in good faith
- DOB is provided in reasonable format

5.2 CONSTRAINTS

TECHNICAL CONSTRAINTS:

- Groq API key required for full functionality
- Tesseract OCR must be installed on system
- Single-threaded document processing (no parallel uploads)
- SQLite not suitable for high-concurrency production

FUNCTIONAL CONSTRAINTS:

- Cannot process documents without text (pure image signatures)
- Cannot verify authenticity of laminated/reflective documents
- Cannot handle handwritten name changes on printed documents
- Vision model context window limits document size

REGULATORY CONSTRAINTS:

- PII storage must comply with local data protection laws
- Audit logs must be retained for minimum 7 years (varies by jurisdiction)
- Aadhaar data handling must comply with UIDAI guidelines
- Human review cannot be bypassed for banking updates

5.3 KNOWN LIMITATIONS

DOCUMENT PROCESSING:

1. Low-quality images: Blurry or dark images may fail extraction
2. Non-standard layouts: Regional Aadhaar format variations
3. Language limitations: Best performance on English text
4. No watermark detection: Cannot verify holographic features
5. Partial OCR failures: Some characters may be misread

AI SCORING:

1. Confidence scores are estimates, not guarantees
2. Rule-based fallback less accurate than LLM scoring
3. Cannot detect sophisticated forgeries
4. May be biased toward certain name patterns

5. New/unusual names may receive lower scores

SCALABILITY:

1. SQLite concurrent write limitations
2. No request queuing or prioritization
3. No load balancing or caching
4. Single point of failure (no redundancy)

INTEGRATION:

1. RPS is a mock, not real banking system
2. No notification system for status updates
3. No bulk import/export functionality
4. No role-based access control (prototype only)

5.4 FUTURE ENHANCEMENTS

RECOMMENDED IMPROVEMENTS:

1. Add support for PASSPORT, PAN, VOTER_ID documents
2. Implement multi-document cross-referencing
3. Add fraud detection ML model
4. Implement request queuing and prioritization
5. Add webhook notifications for status changes
6. Integrate with real banking core systems
7. Add RBAC and audit log archival
8. Implement request caching and rate limiting
9. Add multi-language support for regional documents
10. Implement document versioning and comparison